

AI Adoption in Software Development: Evidence from Developers

Mark Looi¹

This is a preprint version of a manuscript. An earlier version of this study was posted on arXiv under the title [“AI Tools in Software Development: Developer Perceptions and Usage Patterns”](#). The present working paper substantially revises the framing, interpretation, and presentation of the study and supersedes the earlier version.

© 2026 Mark Looi. Licensed under CC BY 4.0.

¹ Mark Looi, 3828 Cascadia Avenue South, Ste 101, Seattle, WA 98118. +1 425 941 2378.
marklooi@looiconsulting.com.

Abstract

AI tools are increasingly used across software engineering, raising questions about how developers adopt and integrate them into everyday practice, and how they perceive the future role of AI in software. A cross-sectional survey of 147 professional software developers was conducted. Statistical analysis was used to examine associations among AI tool usage breadth, usage frequency, perceived productivity, perceived code quality, workflow integration, future adoption intent, and developers' expectations regarding AI-enabled software capabilities and skills.

Broader and more frequent AI tool usage is positively associated with perceived productivity, while broader usage is associated with perceived code quality and stronger future adoption intent. Perceived productivity and perceived code quality are positively associated, suggesting that developers do not perceive a trade-off between speed and quality in their use of AI tools. AI tool usage is broader in coding than in testing, with higher reported productivity gains in coding-related activities. Regression analysis indicates that future adoption intent is positively associated with AI tool usage frequency and ease of workflow integration, while security concerns exhibit a modest negative association.

The study finds that developers report generally positive but differentiated expectations regarding the future role of AI in software. Clustering analysis identifies three developer segments—Enthusiasts, Pragmatists, and Cautious—which differ in AI tool usage, perceived outcomes, future adoption intent, and expectations regarding AI in software. Developers report stronger expectations for anticipatory applications, multimodal input, retrieval-augmented generation, agentic architectures, and benefits of small language models, while expressing more reserved expectations regarding AI-native and disposable applications.

Keywords: Generative AI, Software Development, Software Engineering, Developer Productivity, Code Quality, Technology Adoption

1 Introduction

Artificial Intelligence (AI) tools are rapidly becoming part of everyday software engineering practice. Understanding how developers adopt and integrate these tools has therefore become an important research question. Software engineering provides a useful context for studying AI adoption because AI tools are already used across multiple stages of the software development lifecycle, including coding, testing, documentation, code review, debugging, and design-related work.

This study uses an exploratory empirical approach to examine developers' reported use of AI tools in software development, focusing on perceived productivity, perceived code quality, workflow integration, and future adoption intent. In addition to current tool adoption, the study examines developers' expectations regarding the future role of AI in software, including AI-enabled application capabilities, interaction models, architectures, model-related technologies, and skills.

The study analyzes survey responses from 147 professional developers. Given the perception-based and cross-sectional nature of the data, the study focuses on identifying patterns of association rather than establishing causal relationships.

The findings characterize AI adoption as a workflow-specific and developer-specific phenomenon. They also show that developers' expectations regarding the future role of AI in software extend beyond current tool adoption to include AI-enabled capabilities, architectures, interaction models, and skills.

This paper makes the following contributions:

- Provides an empirical characterization of AI adoption in software development through observed associations among AI tool usage, perceived value, workflow integration, and future adoption intent.
- Shows that developers reporting broader AI use also report higher perceived productivity and perceived code quality, with no perceived trade-off between the two.
- Identifies a testing gap, suggesting that AI adoption differs across software development workflows rather than progressing uniformly across the development lifecycle.
- Identifies three empirically derived developer segments (Enthusiasts, Pragmatists, and Cautious users), demonstrating that AI adoption varies systematically across developers in terms of AI usage, perceived value, future adoption intent, and expectations regarding AI in software.
- Provides empirical evidence regarding developers' expectations about the future role of AI in software, including AI-enabled application capabilities, interaction models, architectures, model-related technologies, and skills.

2 Background and Literature Review

This literature review situates the study within prior research on AI-assisted software development, developer perceptions of AI, and technology adoption in software engineering.

2.1 Empirical Studies of AI Tool Use in Software Engineering

Research on AI-assisted software development has examined both software outcomes and developer experiences. Objective studies have reported mixed findings regarding the effects of AI-assisted development on productivity and code quality (He et al., 2025; Becker et al., 2025).

Survey-based studies examine how developers perceive and use AI tools in practice. Prior work reports that developers often perceive AI tools as beneficial for productivity and workflow support, while also expressing concerns related to correctness, trust, and integration into existing workflows (Liang et al., 2024; Russo, 2024; Sergeyuk et al., 2025; Vaithilingam et al., 2022).

Together, these studies show that AI-assisted software development has been examined through both software artifacts and developer experiences. While objective studies evaluate software outcomes, survey-based studies provide insight into how developers adopt AI tools in everyday practice. The present study builds on the latter perspective by examining patterns of AI tool usage, perceived outcomes, and future adoption among professional developers.

2.2 Technology Adoption and Perceived Utility

Research on technology adoption emphasizes the importance of perceived usefulness and perceived ease of use in shaping system adoption behavior (Davis, 1989; Venkatesh et al., 2003). In AI-assisted software

development contexts, developers' perceptions of usefulness, reliability, and workflow fit may influence both current usage patterns and future adoption intent.

Prior work has identified security risks associated with AI-assisted code generation, including the need for additional verification processes (Negri-Ribalta et al., 2024). Related concerns, such as copyright and intellectual property risk, may also affect how developers evaluate and integrate AI tools into software development workflows.

These perspectives motivate examining how perceived value, workflow fit, and risk relate to AI adoption dynamics within software development practice.

2.3 Adoption Frameworks and Developer Behavior

Diffusion of innovations theory has also been used to describe how technology adoption varies across populations and evolves over time (Rogers, 2003). In AI-assisted development contexts, prior work highlights the potential influence of organizational context, developer cognition, and workflow integration on adoption behavior (Russo, 2024).

These adoption frameworks provide a useful lens for interpreting differences in developer adoption of AI tools and suggest that developers may differ in how they evaluate, incorporate, and continue using emerging technologies.

2.4 Segmentation and Outlook

Prior work suggests that AI-assisted software development is not adopted uniformly across developers or workflows (Liang et al., 2024; Russo, 2024; Sergeyuk et al., 2025). Diffusion-oriented perspectives also suggest that emerging technologies are often adopted unevenly across populations, with different groups exhibiting distinct patterns of adoption, evaluation, and continued use (Rogers, 2003). In this study, segmentation is therefore used to explore whether observed usage and perception measures reveal empirically distinguishable developer groups. The resulting segments are exploratory and empirically derived rather than predefined adopter categories.

Prior research also suggests that generative AI may affect software development beyond code generation by changing the kinds of applications developers build. Recent work on AI-native applications argues that emerging AI-enabled systems may differ from conventional software through their probabilistic behavior, orchestration layers, vector databases, and AI-specific observability requirements (Cao et al., 2025). These developments motivate examining developers' expectations regarding the future role of AI in software, including application capabilities, architectures, interaction models, model-related technologies, and skills.

2.5 Study Focus

This study adopts an exploratory empirical approach to examine AI adoption in software development and developers' expectations regarding the future role of AI in software. It focuses on three related areas: current AI tool usage across coding- and testing-related workflows; associations among AI tool usage, perceived productivity, perceived code quality, workflow integration, contextual adoption factors, and future adoption intent; and developers' expectations regarding future AI-enabled applications, interaction

models, architectures, model-related technologies, and skills. The study also examines whether empirically distinguishable developer segments emerge from usage and perception measures.

3 Research Questions (RQs)

3.1 Current State of AI Use

RQ1: What are the patterns of developers' AI tool usage across coding and testing?

3.2 Usage and Perception

RQ2: How is AI tool usage associated with perceived productivity and perceived code quality?

3.3 Perception and Increased Usage

RQ3: How are perceived productivity and perceived code quality associated with developers' intent to increase AI tool usage?

3.4 Contextual Factors and Adoption Intent

RQ4: How are contextual factors such as integration effort, security concerns, and organizational policy associated with future AI adoption intent?

3.5 Segmentation

RQ5: Do distinct developer segments emerge based on AI tool usage, perceived outcomes, strategic outlook, and future adoption intent?

3.6 The Future of AI in Software

RQ6: How do developers perceive the future role of AI in software, including emerging application paradigms, architectures, interaction models, skills, and model-related technologies?

4 Methodology

4.1 Research Context

This study examines developers' reported use of AI tools in software development practice, with a focus on perceived productivity, perceived code quality, future adoption intent, and expectations regarding the future role of AI in software. The analysis uses developers' perceptions rather than objective measurements of software artifacts or development outcomes.

4.2 Survey Design and Measures

Data was collected using a structured survey of 34 questions, including several multi-item and matrix questions capturing multiple variables.

Most perceptual items were measured using 5-point Likert scales (full question wording and response options are provided in the supplementary material, “Survey of AI in Software - Google Forms.pdf”). The survey captured:

- AI tool usage (frequency and breadth across coding and testing tasks)
- Perceived productivity in coding
- Perceived productivity in testing
- Perceived code quality
- Adoption intent, measured as likelihood of increased AI usage
- Contextual factors, including integration effort, cost, security, and policy considerations
- Strategic outlook regarding the future role of AI in software, including AI-enabled application capabilities, interaction models, architectures, model-related technologies, and skills.

4.3 Variables and Indices

To reduce dimensionality and improve reliability, related survey items were aggregated into composite indices. Indices were calculated using either summation or averaging of Likert-scale responses, depending on the construct. The primary indices used in the analysis were:

- Intent to Increase Usage Index (dependent variable)
- Perceived Quality Index (mean of coding and testing perceived quality)
- AI Coding Tool Index (breadth of coding tool usage)
- AI Testing Tool Index (breadth of testing tool usage)
- Mean Concern Index (perceived challenges of the cost of implementation and licensing, security and compliance, and the risk of copyright or IP infringement)
- Strategic Outlook Index (forward-looking beliefs about future of AI in software)

The Strategic Outlook Index measures respondents’ expectations regarding the future evolution of AI-enabled software applications and architectures, including AI-native applications, conversational and multimodal interfaces, agentic systems, retrieval-augmented generation, and emerging model ecosystems. The internal consistency of composite measures was assessed using Cronbach’s alpha, with values indicating acceptable reliability for exploratory research (Taber, 2018). For more details on the composition of the indices, see Appendix Table 2: Composite Indices Definitions.

4.4 Data Collection and Participants

The survey was distributed to software professionals through email and social media channels associated with the author during October and November 2025. A total of 147 valid responses from individuals actively engaged in software development activities were included in the analysis.

Participants represented a range of roles, experience levels, and geographic regions (see Supplement Appendix Table 1: Survey Demographics). Participation in the survey was voluntary and no personally identifiable information was collected. Respondents could choose not to participate or discontinue at any time. Completion of the survey was taken as implied consent, consistent with minimal-risk research practices.

The full survey instrument and cleaned dataset are provided in the supplementary material.

4.5 Statistical Analysis

Given the ordinal nature of the survey data, Spearman rank correlation was used to examine associations between variables. The research questions were addressed as follows:

- RQ1: descriptive statistics were used to characterize AI tool usage patterns across coding and testing activities.
- RQ2–RQ3: Spearman correlations were used to examine associations among AI tool usage, perceived productivity, perceived code quality, and future adoption intent.
- RQ4: linear regression was used to examine associations between contextual factors and future adoption intent.
- RQ5: principal component analysis (PCA) followed by K-means clustering were used to identify potential developer segments based on AI usage and perception measures.²
- RQ6: descriptive statistics were used to characterize developers' expectations regarding future AI-enabled software capabilities, architectures, interaction models, model-related technologies, and skills.

Statistical significance was evaluated at $\alpha < 0.05$. Because the study is exploratory and cross-sectional, the analyses are intended to characterize patterns of association rather than establish causal relationships.

5 Findings

5.1 RQ1: Current AI Usage

AI tools are widely used across software development activities, particularly in coding.

Sixty-seven percent of respondents report using AI coding tools “often” or “always”, with high perceived utility for coding (Table 1). Supporting details are provided in the Supplement, Figure 1.

Usage breadth differs across domains. Developers engage in a median of 5 coding-related AI activities (out of 11), while testing usage is more limited (median = 2 activities).

Perceived utility of AI tools is generally high, with more favorable ratings for coding than testing activities (Table 1).

Metric	Area	Mean (1-5)	% "High/Very High"
Accuracy/Relevance	Coding	4.17	53%
Bug-Finding Effectiveness	Testing	3.36	43%

Table 1: Developer Ratings of Utility

² For the PCA, the number of clusters ($k = 3$) was selected based on inspection of the within-cluster sum of squares curve and interpretability of the resulting clusters. Silhouette analysis indicated slightly higher scores for $k = 2$, but this solution produced overly coarse groupings. The $k = 3$ solution provided a better balance between cluster separation and interpretability. See 12.1.

Usage patterns for coding activities show both substantial usage and variation in breadth (Figure 1). The median developer engages in 5 activities, with nearly half (49%) using 6 or more, while 23% engage in 3 or fewer. Code completion is the most common use case (67%), see Supplement Figure 2.

Distribution of AI Coding Activities

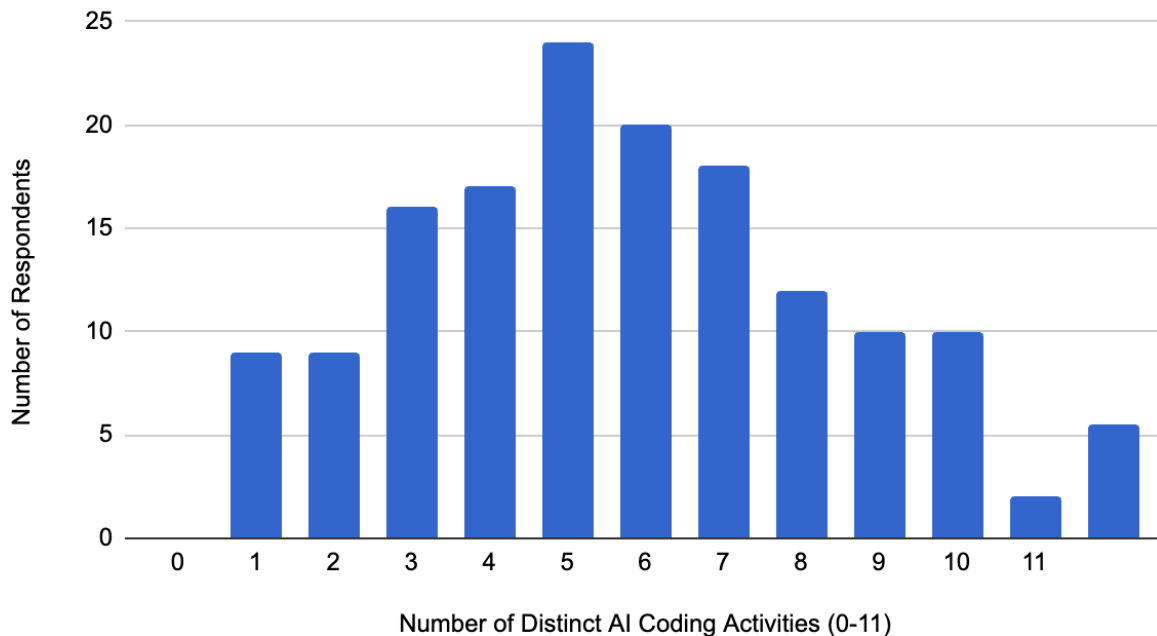


Figure 1: Distribution of AI Coding Activities

In contrast, testing usage is more limited and concentrated (Figure 2). While 68% of developers report using AI testing tools and 43% say they use them “Often” or “Always”, the mean number of activities is 2.29 (median = 2). Over one-third (35%) engage in one or no testing-related AI activities, although 38.9% report using three or more. Test case generation is the most common testing use case (72%), followed by test automation (60%), while other activities are less widely used. Supporting details are provided in the Supplement, Figures 6 and 7.

Distribution of AI Testing Activities

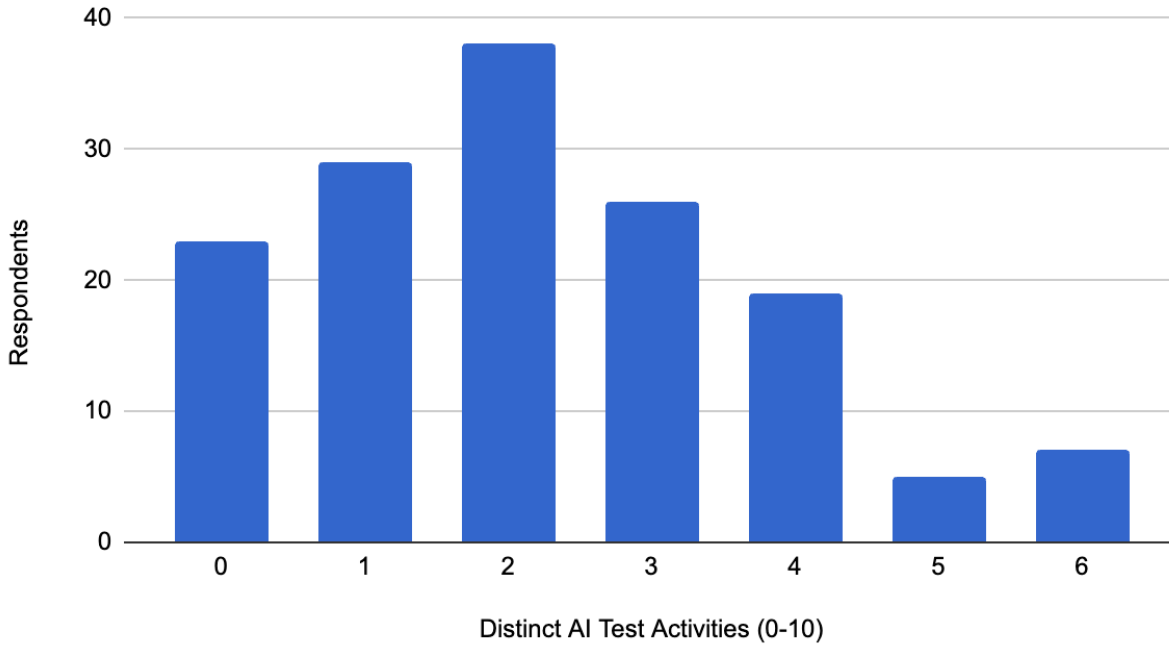


Figure 2: Distribution of AI Testing Activities

These workflow-level usage patterns provide the basis for examining how AI tool usage relates to perceived productivity, perceived code quality, and continued AI adoption.

5.2 RQ2: Associations with Perceived Productivity and Quality

AI tool usage is positively associated with perceived productivity in both coding and testing contexts, with stronger relationships observed for coding activities. Developers report higher productivity gains for coding than for testing (Table 2) with the median for coding twice that of testing.

Perceived Productivity Measure	% Reporting > 3 Hours Saved	Mean (1–6)	Median (1–6)
Coding activities	60%	3.92	4
Testing activities	48%	3.14	2

Table 2: Perceived Productivity (PP)

Most developers report that AI tools have a positive impact on perceived code quality. Approximately 74% of respondents report improvements in code quality, while 68% report improvements in testing contexts. Details are provided in the Supplement, Figures 5 and 10.

Correlation analysis shows that both frequency and breadth of AI tool use are positively associated with perceived productivity in coding and testing (Table 3). Frequency of use exhibits the strongest associations, while breadth of use (as captured by the AI Coding Tool Index and AI Testing Tool Index) is also positively related to perceived productivity.

Variable Pair	N	ρ	p-value
AI Coding Tool Index – Perceived productivity code	145	0.347	< 0.001
Frequency of AI coding tool use – Perceived productivity code	147	0.458	< 0.001
AI Testing Tool Index – Perceived productivity test	132	0.357	< 0.001
Frequency of AI testing tool use – Perceived productivity test	132	0.479	< 0.001
PQI – Perceived productivity code	147	0.190	0.021
PQI – Perceived productivity test	132	0.358	< 0.001

Table 3: *Perceived Productivity Findings (Note: N varies because not all Developers used AI testing tools.)*

Perceived productivity is also positively associated with perceived code quality, with a weaker association observed for coding-related activities and a stronger association observed for testing-related activities.

Higher breadth of AI tool usage is also positively associated with perceived code quality. As shown in Table 4, both coding and testing tool breadth exhibit weak but statistically significant correlations with PQI.

Variable Pair	N	ρ	p
AI Coding Tool Index – PQI	132	0.2356	0.00407
AI Testing Tool Index – PQI	132	0.289	0.00039

Table 4: *Breadth and Perceived Quality Findings*

Overall, the findings for RQ2 indicate that broader and more frequent AI tool usage is positively associated with perceived productivity, while perceived productivity and perceived code quality are also positively associated. These relationships are stronger in coding-related activities than in testing-related activities.

These findings provide the basis for examining how perceived outcomes relate to future adoption intent.

5.3 RQ3: Associations Between Perceived Outcomes and Adoption Intent

Developers report a high overall intent to increase their use of AI tools, particularly for coding (Table 5).

Category	N	Mean	Median
Coding Tools	147	4.32	5
Testing Tools	145	3.89	4
Composite Index	147	4.08	4

Table 5: *Intent to Increase Usage of AI Tools*

Perceived productivity, perceived code quality, and breadth of AI tool usage are all positively associated with future adoption intent (Table 6).

Variable Pair	N	ρ	p-value
---------------	---	--------	---------

Perceived productivity in testing – Intent	132	0.319	< 0.001
Perceived productivity in coding – Intent	147	0.227	0.006
PQI – Intent	147	0.231	0.005
AI Coding Tool Index – Intent	147	0.317	< 0.001
AI Testing Tool Index – Intent	132	0.347	< 0.001

Table 6: Associations with Adoption Intent

Overall, the findings for RQ3 indicate that future adoption intent is positively associated with perceived productivity, perceived code quality, and broader AI tool usage. The strongest associations are observed for testing-related productivity and testing-related breadth of usage.

5.4 RQ4: Associations with Contextual Factors and Adoption Intent

To examine associations between contextual factors and future adoption intent when considered jointly, a linear regression model was estimated using the Intent to Increase Usage Index as the dependent variable.

The model included:

- Frequency of AI testing tool use
- Perceived quality (PQI)
- Ease of integration
- Cost
- Security concerns
- Organizational policy
- Developer interest

The model explains 29.5% of the variance in adoption intent ($R^2 = 0.295$, $F(7, 138) = 8.14$, $p < 0.001$).

Variable	β	p-value
Frequency of AI testing tool use	0.38	< 0.001
Ease of integration	0.29	0.001
Security concerns	-0.17	0.001
Cost	n.s.	
PQI	n.s.	
Organizational policy	n.s.	
Developer interest	n.s.	

Table 7: Regression Results Predicting Adoption Intent

Frequency of AI tool use and ease of integration are positively associated with adoption intent, while security concerns are negatively associated. Cost, perceived quality, organizational policy, and developer interest are not statistically significant in this model.

In this model, future adoption intent is most clearly associated with AI tool usage frequency and ease of workflow integration, while cost, perceived quality, organizational policy, and developer interest are not statistically significant predictors.

5.5 RQ5: Developer Segmentation

To examine how usage, perceived outcomes, and adoption-related measures relate across developers, a clustering analysis was performed using four variables: Intent to Increase Usage, AI Coding Tool Index (usage breadth), Strategic Outlook Index, and the Perceived Quality Index (PQI). Principal component analysis was applied for dimensionality reduction, followed by K-means clustering. A three-cluster solution was selected based on interpretability and stability, although alternative segmentations are plausible.

The analysis identifies three developer segments:

- Enthusiasts (n = 56): High usage breadth, higher perceived productivity and perceived quality, and high intent to increase usage.
- Pragmatists (n = 53): Moderate AI usage breadth and lower perceived quality, but high future adoption intent and Strategic Outlook scores comparable to Enthusiasts.
- Cautious (n = 38): Lower usage breadth, weaker perceived outcomes, and lower intent to increase usage.

Enthusiasts report the highest AI usage breadth and perceived quality. Pragmatists occupy an intermediate position in AI usage breadth and report lower perceived quality, but their future adoption intent and Strategic Outlook scores are comparable to Enthusiasts. Cautious developers report lower usage breadth, weaker perceived outcomes, lower future adoption intent, and more reserved strategic outlook.

The clustering variables capture approximately 42% of the variance in the data, suggesting that these dimensions account for a substantial portion of the variation in developer perceptions and usage patterns.

	Enthusiasts	Cautious	Pragmatists	Total/Average
Number	56	38	53	147
Intent to Increase Usage Index	4.49	2.87	4.51	
Strategic Outlook Index	3.77	2.74	3.77	
AI Coding Tool Index (Usage Breadth)	7.12	3.74	5.21	
PQI	4.69	3.66	3.40	
Mean Concern Index Score (1-5)	3.18	3.46	3.22	3.27
AI Coding Policy (Yes)	33	2	14	49
AI Coding Policy (No)	23	36	39	98
Policy Support %	58.9%	5.3%	26.4%	33.3%

Table 8: Developer Segments Identified by Clustering

To further explore organizational context, the presence of AI-related development policies was compared across clusters. A chi-square test indicates a significant association between cluster membership and

policy presence ($\chi^2 = 45.59$, $df = 2$, $p < 0.001$). Enthusiasts are more likely to report the presence of organizational AI coding policies (59%) compared to Pragmatists (26%) and Cautious developers (5.3%).

This association suggests that organizational context and developer segmentation are related, although the direction of this relationship cannot be determined from the present data.

Overall, the segmentation suggests that Enthusiasts stand apart through broader AI tool usage, higher perceived productivity and quality, and stronger future adoption intent. Pragmatists report moderate AI usage breadth and relatively critical perceived quality evaluations, while still reporting strong future adoption intent and strategic outlook regarding AI in software. The Cautious group exhibits lower adoption, weaker perceived outcomes, and more reserved expectations.

5.6 RQ6: Developer Perspectives on the Future of AI in Software

The survey examined respondents' expectations regarding the future role of AI in software, including anticipated application capabilities, interaction models, software architectures, model-related technologies, and skills required to work with AI-enabled systems. Developers report a generally positive but nuanced outlook regarding AI both as a development aid and as an embedded capability in future software systems. Expectations for continued AI integration coexist with recognition of technical, organizational, and engineering challenges.

In terms of future capabilities, respondents expressed the strongest expectations regarding applications that anticipate user needs, multimodal input, benefits of small language models, retrieval-augmented generation, and agentic architectures. Expectations were more moderate for dynamically generated user interfaces, hybrid models, and conversational interfaces, and lower for the current development status of AI-native applications and disposable applications (Table 9).

Developer segment results show that Enthusiasts and Pragmatists both reported relatively positive expectations regarding the future role of AI in software. Pragmatists reported means equal to or higher than Enthusiasts for several items, including conversational interfaces, multimodal input, agentic architectures, retrieval-augmented generation, and benefits of SLMs. This suggests that the Pragmatist segment is not skeptical about the future role of AI in software, even though it differs from Enthusiasts in current tool use and perceived quality.

The Cautious segment reported lower means across all Strategic Outlook Index components. However, these respondents were not uniformly dismissive. Their highest ratings were for applications that anticipate user needs and benefits of SLMs, while their lowest ratings were for AI-native applications, disposable applications, conversational interfaces, and dynamically generated user interfaces.

Strategic Outlook Index Variable	Overall (N=135–145)	Enthusiasts (N=53–56)	Pragmatists (N=48–53)	Cautious (N=34–37)
Primary UI will be conversational	3.48	3.64	3.87	2.70
UI will be generated dynamically by AI	3.60	3.95	3.83	2.76
Apps use multimodal input	3.83	4.09	4.19	2.92

Apps anticipate user needs	4.06	4.30	4.23	3.43
Development status of AI-native apps	2.92	3.30	3.17	2.00
Agentic architecture importance	3.62	3.78	4.04	2.77
RAG importance	3.68	3.89	4.16	2.72
Disposable applications importance	2.78	3.17	2.73	2.25
SLMs importance	3.29	3.56	3.37	2.78
Hybrid models (LLM + Physical)	3.55	3.77	3.71	2.97
Benefits of SLMs	3.81	3.96	4.04	3.24

Table 9: Developer Expectations Regarding the Future of AI in Software³

Developers also identify several challenges associated with AI-native systems. Model hallucinations, security risks, and testing complexity are all rated as challenges (see Appendix Table 4). Skills related to orchestration, data management, and prompt engineering are consistently rated as important (Table 10).

Future Developer Skill Importance	N	Mean	Median
Importance of Orchestration Workflow Design	142	3.98	4
Importance of Data management	142	3.9	4
Importance of Prompt Engineering	144	3.9	4
Importance of Model Selection	142	3.54	4
Importance of Foundation model fine tuning	143	3.41	3

Table 10: Future Skills

The composite Strategic Outlook Index was used as a summary measure for examining associations with other constructs. It is positively associated with perceived importance of orchestration ($\rho = 0.406$), AI-native development status ($\rho = 0.427$), and intent to increase AI usage ($\rho = 0.305$), with weaker positive associations observed for perceived productivity measures (Appendix Table 3).

6 Discussion

The central finding of this study is that AI adoption in software development is characterized by associations among AI tool usage, perceived productivity, perceived code quality, workflow integration,

³ Values are mean responses on a 1–5 scale. Segment means are based on the K-means clusters reported in 5.5. Because not all respondents answered every strategic outlook question, means were calculated from available item-level responses within each segment; valid N therefore varies by cell. No imputation was used.

and future adoption intent, while also being accompanied by differentiated expectations regarding the future role of AI in software. These relationships vary across development workflows and developer segments.

6.1 Early Patterns of AI Adoption Across Workflows (RQ1–RQ2)

The findings indicate that AI tools are widely used across software development activities, particularly in coding, with lower but still substantial use in testing. Usage spans multiple tasks, although the breadth of application differs across coding and testing contexts.

AI tool usage is positively associated with perceived productivity in both coding- and testing-related activities. These associations are stronger for coding activities, where respondents report greater time savings and broader tool usage. A notable gap exists between coding and testing in both usage breadth and perceived productivity. While many developers report engaging with multiple coding-related AI activities, testing usage tends to be concentrated in fewer tasks.

The coding-testing gap indicates that AI is integrated differently across software development workflows, reinforcing the view that adoption should be understood at the level of work practices.

These findings are broadly consistent with prior work reporting perceived productivity gains from AI-assisted development (Bird et al., 2023; Liang et al., 2024; Vaithilingam, 2022). However, the present study focuses on developers' perceptions and experiences rather than objective productivity outcomes. The results therefore should not be interpreted as evidence of objective productivity improvements or declines.

6.2 Perceived Productivity and Quality (RQ2)

The findings show that usage breadth and frequency of AI tool use are positively associated with perceptions of increased productivity in both coding and testing. Perceived productivity and perceived code quality are also positively associated. Together, these relationships suggest that broader usage, higher perceived productivity, and higher perceived quality tend to co-occur in developers' experience.

This pattern differs from objective studies reporting potential quality concerns (He et al., 2025) or productivity trade-offs in AI-assisted development (Becker et al., 2025). The present findings do not directly support or contradict those studies, as this study examines developers' perceptions rather than objective software outcomes.

The distinction between objective outcomes and developer perceptions remains important because perceived usefulness is a primary determinant of technology adoption and system use (Venkatesh et al., 2003). In the current study, developers who report broader AI tool usage also tend to report more favorable perceived outcomes overall, including higher perceived productivity and perceived code quality. These associations suggest that perceived productivity and perceived quality form part of the broader set of factors associated with developers' perceived value from AI-assisted software development.

6.3 Adoption Intent and Contextual Factors (RQ3–RQ4)

Developers report a high intention to increase their future use of AI tools, particularly for coding-related activities. Future adoption intent is positively associated with broader AI tool usage, perceived productivity, and perceived code quality. Together, these findings suggest that developers who use AI tools more extensively and perceive greater productivity and quality benefits are also more likely to intend continued AI adoption.

Regression analysis further identifies which contextual factors remain associated with future adoption intent when considered jointly. AI tool usage frequency and ease of workflow integration exhibit positive associations with future adoption intent, while security concerns exhibit a modest negative association. Cost and organizational policy do not show statistically significant associations in this model. The negative association between security concerns and future adoption intent is consistent with prior work emphasizing the need for security verification and caution when incorporating AI-generated code into software development processes (Negri-Ribalta et al., 2024).

These findings are broadly consistent with technology adoption research emphasizing perceived usefulness and ease of use as important correlates of adoption behavior (Davis, 1989; Venkatesh et al., 2003).

6.4 Developer Segmentation (RQ5)

The clustering analysis identifies heterogeneity in developers' AI tool usage, perceived outcomes, and future adoption intent. The identified segments—Enthusiasts, Pragmatists, and Cautious developers—differ in usage breadth, perceived productivity, perceived quality, and intended future AI tool usage (Table 8).

These patterns are broadly consistent with diffusion-oriented perspectives on technology adoption, in which different groups exhibit differing levels of adoption of emerging technologies (Rogers, 2003).

The identified segments illustrate that AI adoption is heterogeneous within development teams. Developers differ not only in the breadth of AI tool usage but also in their perceptions of its value, future use, and strategic importance.

The presence of organizational AI policies is also associated with developer segments, with higher reported policy prevalence among the more active AI adopters. However, the cross-sectional design does not allow the direction of this relationship to be determined. Organizations may adopt AI policies in response to increased AI use, policies may encourage broader AI adoption, or both may reflect broader organizational AI maturity.

6.5 Strategic Outlook

The future-oriented results broaden the interpretation of AI adoption beyond current tool use. Strategic outlook captures developers' expectations about the future role of AI in software, including application capabilities, interaction models, architectures, model ecosystems, and skills. This makes it conceptually different from adoption intent: adoption intent concerns whether developers expect to use AI tools more, while strategic outlook concerns how developers believe software itself may evolve.

This distinction is important because recent work on AI-native applications suggests that AI-enabled systems may differ from conventional software through probabilistic behavior, orchestration layers, vector databases, and AI-specific observability requirements (Cao et al., 2025). The present findings provide developer-level evidence that these future-oriented issues are salient to practitioners, although respondents distinguish among different possible directions rather than expressing uniform optimism.

Respondents report stronger expectations for some developments and more reserved expectations for others. This suggests that developers are evaluating specific future capabilities and architectural patterns differently. The strongest expectations are not simply about using AI assistants more, but about how AI may become embedded in application behavior, interaction models, and software architectures.

The segment-level results further refine this interpretation. Enthusiasts and Pragmatists both report relatively positive expectations regarding the future role of AI in software, while the Cautious segment reports lower expectations overall. The Pragmatist segment is especially important because it shows that measured current adoption does not necessarily imply skepticism about the future of AI-enabled software. Developers may be selective or cautious adopters today while still holding relatively strong expectations about future AI-enabled applications, architectures, and interaction models.

The item-level and segment-level findings show that future-oriented expectations regarding AI in software are related to, but not reducible to, current AI adoption and perceived outcomes.

7 Implications of the Research

This study provides practical implications for organizations and individual developers navigating both current AI tool adoption and the broader incorporation of AI into software systems, architectures, and development practices.

7.1 Implications for Development Teams and Management

These findings suggest that organizations should view AI adoption as an ongoing capability-building effort. Sustained value may depend not only on providing access to AI assistants, but also on supporting experimentation, workflow integration, and the development of skills needed for AI-enabled software systems, including orchestration, data management, evaluation, and governance.

7.1.1 Integration and Usage Patterns

Broader AI tool usage, perceived productivity, perceived quality, and future adoption intent are positively associated, suggesting that organizations may benefit from supporting continued experimentation and integration of AI across development activities rather than limiting usage to isolated tasks.

Ease of integration is also positively associated with future adoption intent. Tools that align with existing workflows and reduce friction may therefore be more likely to support sustained developer use.

7.1.2 Skills and Architectural Orientation

Developers report relatively high perceived importance for orchestration-related skills, data management, prompt engineering, retrieval-augmented generation, and agentic systems. These findings suggest that organizations should not treat AI capability solely as proficiency with coding assistants. Developers may increasingly need to understand how AI capabilities are incorporated into software systems through data

pipelines, retrieval architectures, model orchestration, evaluation workflows, and human-in-the-loop controls.

Organizations may therefore benefit from aligning training and professional development efforts with these emerging areas, particularly where AI is expected to become part of product functionality rather than just a development aid.

7.1.3 Developer Segmentation

The identified developer segments suggest that AI tool usage and future expectations are not uniform across development teams. Organizations may therefore benefit from recognizing differences in developer experience, experimentation, perceived quality, and expectations regarding AI-enabled software rather than assuming a single adoption pattern across all users.

Enthusiasts may help demonstrate effective AI use and support knowledge sharing within teams. Pragmatists may be especially important because they appear receptive to the future role of AI in software while remaining more measured in current adoption or perceived quality. Their feedback may therefore be useful for identifying where tools, workflows, or governance practices need improvement. Cautious developers may require clearer guidance, training, security assurances, and opportunities for low-risk experimentation before broader adoption occurs.

These findings also suggest that organizational policies and governance structures may need to evolve alongside differing levels of AI usage and differing expectations about AI-enabled software within teams.

7.1.4 Adoption Context

Future adoption intent is positively associated with AI usage frequency and ease of workflow integration, while cost and organizational policy are not statistically significant predictors. Organizations may therefore benefit from focusing on reducing friction in AI adoption and supporting integration into existing development practices.

Security concerns exhibit a modest negative association with future adoption intent, suggesting that clear governance, security guidance, and policy visibility may help support adoption among more cautious developers.

7.2 Implications for Developers and Professionals

7.2.1 Usage and Workflow Integration

Higher frequency and broader use of AI tools are associated with more favorable perceptions of productivity and quality. Developers may therefore benefit from exploring AI adoption across a wider range of development activities, including documentation, testing, and code review in addition to code generation.

The results indicate that developers place relatively high importance on prompt design, orchestration, and data integration. For individual developers, the findings suggest that AI literacy should extend beyond familiarity with coding assistants. Developers may benefit from developing practical fluency in how AI capabilities are incorporated into software systems, including retrieval-augmented generation, orchestration workflows, agentic patterns, data management, and evaluation practices.

7.2.2 Reflecting on Segmentation

The identified developer segments highlight that usage patterns and perceptions of AI tools vary. Understanding these segments may help developers reflect on their own AI usage, perceived value, and future expectations to guide decisions for skill development and tool adoption.

The Pragmatist segment suggests that developers can be critical about AI tools while recognizing their future importance. A measured approach that combines experimentation with quality awareness, verification, and selective use may be particularly valuable as AI becomes more deeply embedded in software systems.

For developers who are more cautious, the findings suggest the value of low-risk experimentation and targeted skill development. Skepticism can be valuable when it supports careful review, security awareness, and testing discipline.

More broadly, individual developers may benefit from treating AI as both a productivity tool and a design/architecture concern. Skills in prompt engineering, data management, orchestration, model selection, and evaluation may become increasingly relevant as AI capabilities move from external assistants into the software products and workflows developers build.

7.3 Further Research

Several avenues for future research emerge from the limitations and open questions identified in this analysis.

7.3.1 Measurement of Productivity and Quality

The findings indicate a positive association between perceived productivity and perceived code quality. However, these measures are based on self-reported perceptions.

Prior research has examined objective measures of code quality and productivity in AI-assisted development, with some studies identifying potential trade-offs or declines in measured quality (e.g., He et al., 2025). The present study complements this work by focusing on developer perceptions, which do not reflect such trade-offs in developer experience.

Future research could jointly examine perceived and measured outcomes, exploring whether productivity improvements are accompanied by changes in defect rates, maintainability, or technical debt, and how these perceptions evolve over time. Longitudinal studies could help determine whether such discrepancies persist, diminish, or evolve as tools and development practices mature.

7.3.2 Temporal Dynamics and Work Reallocation

The cross-sectional design of this study does not capture how productivity gains evolve over time, how time savings are reallocated, or how developers' future-oriented expectations regarding AI in software change with experience.

Future studies could use longitudinal methods or experience sampling approaches to examine productivity effects more precisely and to investigate how developers use time saved through AI assistance. This could include examining whether gains are offset by additional review, correction, or integration effort, or whether time is redirected toward higher-value activities such as system design or architectural work.

Similar methods could also track how developers' strategic outlook evolves as AI tools and AI-enabled software capabilities mature.

AI-assisted development may also complicate traditional approaches to measuring software engineering productivity and quality. Iterative prompting, conversational debugging, architectural exploration, and discarded generations may not be fully visible through repository-level artifacts or commit-based metrics alone. Future research combining perception-based methods, workflow telemetry, repository analysis, and longitudinal observation may provide a more comprehensive understanding of AI-assisted development practices.

7.3.3 Post-Adoption Effects and Hidden Costs

While perceived concerns such as cost and security show limited association with adoption intent in this study, post-adoption effects remain an open question.

Future research could examine whether AI-assisted development introduces additional forms of rework, such as verifying code provenance or checking correctness of generated code, and whether these offset perceived productivity gains. Understanding these dynamics would provide a more complete picture of the net impact of AI tools in practice.

7.3.4 A Gap between Coding and Testing

The observed difference in usage breadth between coding and testing suggests an area for further investigation. Future studies could examine the underlying causes of this imbalance, including differences in tool maturity, task complexity, organizational constraints, or developer confidence in testing-related AI tools. Understanding these factors may help explain variation in adoption across different stages of the development lifecycle.

7.3.5 Developer Segmentation and Adoption Trajectories

The identification of developer segments based on usage, perceptions, and outlook suggests a potential framework for understanding variation in adoption behavior.

Longitudinal research could examine how these segments evolve over time, including whether developers transition between segments as their experience with AI tools changes. Such studies could also assess whether segment membership is associated with differences in long-term adoption patterns or outcomes.

7.3.6 Organizational Context and Policy

The association between organizational policy presence and developer segments raises questions about the role of governance in AI adoption.

Future research employing longitudinal or quasi-experimental designs could examine the directionality of this relationship. For example, studies could investigate whether policy development follows increased adoption, or whether formal policies influence subsequent usage patterns, or whether both are shaped by underlying organizational factors.

7.4 Threats to Validity and Limitations

This study provides empirical evidence on AI tool usage, perceived productivity, perceived quality, and future-oriented expectations. However, several limitations should be considered when interpreting the

findings relating to internal, external, and construct validity. As with all cross-sectional survey studies, the observed relationships should be interpreted as associative rather than causal.

7.4.1 Internal Validity

Internal validity concerns the extent to which observed relationships reflect associations rather than confounding factors.

The study relies on self-reported measures (e.g., time saved, perceived quality, frequency of use). Such measures may be affected by recall bias, social desirability bias, or differences in individual interpretation. As a result, perceived productivity gains and quality assessments may not correspond to objective outcomes.

Independent and dependent variables were collected using the same survey instrument and from the same respondents. This may inflate observed associations due to shared method variance (Common Method Bias).

The observed associations do not establish directionality. For example, while higher AI usage is associated with higher perceived productivity, it is also possible that more experienced or more productive developers are more likely to adopt and use AI tools extensively.

7.4.2 External Validity

External validity relates to the generalizability of the findings.

The study focuses on perceived productivity and quality (i.e., perception-based measures), which may differ from objective measures. As a result, findings may not directly translate to environments where objective performance metrics are the primary concern.

The sample (N = 147) was drawn from professional networks and voluntary participation, including industry contacts associated with the researcher, which may influence sample composition. While sufficient for statistical analysis, it may not fully represent the broader population of software developers. In particular, respondents may be more likely to have prior exposure to or interest in AI tools.

The AI development landscape is evolving rapidly. Perceptions of tools, risks, and architectural approaches may change over time, limiting the durability of the findings.

7.4.3 Construct Validity

Construct validity refers to how well the measures capture the intended concepts.

Constructs such as perceived productivity and perceived code quality are subjective developer assessments and were measured using ordinal scales. Differences in interpretation across respondents may introduce measurement error and affect the strength of observed associations.

Terms such as “agentic architecture,” “orchestration,” and “RAG” may not be uniformly understood by all respondents. This may introduce noise into measures related to strategic outlook and expectations regarding future AI-enabled software.

Some constructs (e.g., perceived impact of hallucinations) were measured using single survey items. Single-item measures may have lower reliability compared to multi-item scales and may not fully capture the complexity of the underlying concepts.

8 Conclusion

This study provides empirical evidence on AI tool usage, perceived outcomes, future adoption intent, and developers' expectations regarding the future role of AI in software. The findings indicate that AI tools are widely used across software development activities, particularly in coding-related tasks. Broader and more frequent AI tool usage are positively associated with perceived productivity, perceived code quality, and future adoption intent.

The results also suggest that developers do not perceive a simple trade-off between speed and quality in their use of AI tools. Perceived productivity and perceived code quality are positively associated, indicating that respondents who report greater productivity benefits also tend to report more favorable quality perceptions. At the same time, adoption differs across software development workflows, with coding activities currently exhibiting broader use and greater perceived productivity than testing-related activities.

Beyond current tool use, the study finds that developers report generally positive but differentiated expectations regarding the future role of AI in software. Respondents express stronger expectations for anticipatory applications, multimodal input, retrieval-augmented generation, agentic architectures, and benefits of small language models, while reporting more moderate or reserved expectations for AI-native applications, disposable applications, and some interface paradigms. Developers also assign relatively high importance to future skills involving orchestration, data management, prompt engineering, model selection, and evaluation, while recognizing challenges related to hallucinations, security, and testing complexity.

The analysis identifies substantial heterogeneity among developers. The Enthusiast, Pragmatist, and Cautious segments differ in AI tool usage, perceived productivity, perceived code quality, future adoption intent, and expectations regarding AI in software. These empirically derived segments are broadly consistent with diffusion-oriented perspectives on technology adoption while remaining grounded in observed patterns of AI use and perception. The segment-level findings also show that current adoption and strategic outlook are related but not identical: Pragmatists, for example, report relatively strong expectations regarding the future role of AI in software despite differing from Enthusiasts in current use and perceived quality.

The findings characterize AI adoption in software development as both a present workflow phenomenon and a forward-looking shift in how developers understand software systems, architectures, and development skills. Developers making broader use of AI tools report greater perceived value and stronger intentions to expand AI use, but the study also shows that developers' expectations about the future of AI in software are more nuanced than a single measure of adoption intensity. These results suggest that AI's role in software development should be studied not only as a matter of tool adoption, but also as part of a broader evolution in software functionality, architecture, and professional practice.

9 Glossary

Term / Acronym	Definition
AI Coding Tool Index	A composite index reflecting the breadth of AI coding tool usage across 11 specific coding activities (e.g., debugging, refactoring).

AI Testing Tool Index	A composite index reflecting the breadth of AI testing tool usage across 6 specific test activities (e.g., test cases).
Testing Gap	The observed disparity in AI tool usage and perceived productivity between testing and coding activities.
Developer Segments	Empirically derived clusters of developers based on usage patterns, perceived outcomes, strategic outlook, and adoption intent.
Perceived Quality Index (PQI)	Composite measure of perceived code quality.

Table 11: Glossary

10 Declarations

Funding: The authors did not receive support from any organization for the submitted work. No funding was received to assist with the preparation of this manuscript.

Data Statement: An anonymized dataset and data dictionary supporting the findings of this study are available at <https://zenodo.org/records/20045102>. Supplementary materials, including the survey instrument and additional figures, are provided with the submission. This study involved anonymous survey data and did not collect personally identifiable information. Participation was voluntary, and responses were used solely for research purposes. In accordance with institutional guidelines for minimal-risk research, formal ethical approval was not required.

Conflict of Interest: Mark Looi is an independent researcher and the President of Looi Consulting. The author has no relevant financial or non-financial interests to disclose.

During the preparation of this work the author used ChatGPT and Gemini in order to copy edit and construct tables. After using this tool/service, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

11 References

- Becker, Joel, et al. (2025). “Measuring the impact of early-2025 AI on experienced open-source developer productivity.” arXiv preprint arXiv:2507.09089.
- Bird, C., et al. (2023). Taking Flight with Copilot: Early insights and opportunities of AI-powered pair programming tools. *ACMQueue*, 20(6), 35–57. <https://doi.org/10.1145/3582083>
- Cao, L., Li, S., Fan, Y., Li, D., & Zhong, C. (2025). Towards the Next Generation of Software: Insights from Grey Literature on AI-Native Applications. arXiv preprint arXiv:2509.13144.
- Davis, F. D. (1989), “Perceived usefulness, perceived ease of use, and user acceptance of information technology”, *MIS Quarterly*, 13 (3): 319–340, doi:10.2307/249008, JSTOR [249008](https://www.jstor.org/stable/249008).
- He, H., et al. (2025). Does AI-Assisted Coding Deliver? A Difference-in-Differences Study of Cursor’s Impact on Software Projects. *arXiv preprint* arXiv:2511.04427v3.
- Liang, Jenny T., et al. (2024). “A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges.” *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*.

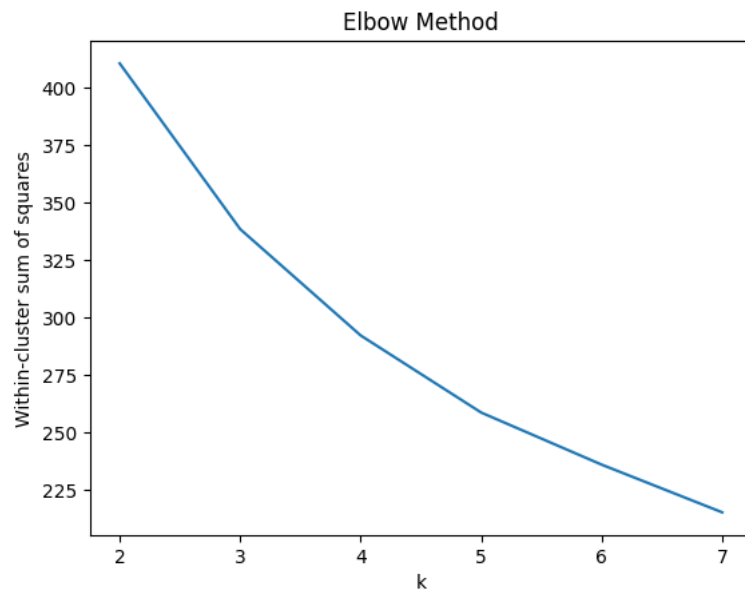
- Meyer, A. N., Barton, L., Murphy, G. C., Zimmermann, T., Fritz, T. (2017). “The work life of developers: Activities, switches and perceived productivity”. IEEE Transactions on Software Engineering. <https://doi.org/10.1109/TSE.2017.2656886>.
- Negri-Ribalta, C., Rémi Geraud-Stewart, Sergeeva, A. and Lenzini, G. (2024). A systematic literature review on the impact of AI models on the security of code generation. *Frontiers in Big Data*, 7. doi: <https://doi.org/10.3389/fdata.2024.1386720>.
- Rogers, E. M. (2003). *Diffusion of innovations* (5th ed.). Free Press.
- Russo, Daniel. (2024). “Navigating the Complexity of Generative AI Adoption in Software Engineering.” *ACM Transactions on Software Engineering and Methodology*.
- Sergeyuk, Aleksandra, et al. (2025). “Using AI-Based Coding Assistants in Practice: State of Affairs, Perceptions, and Ways Forward.” *Information and Software Technology*. <https://doi.org/10.48550/arXiv.2406.07765>
- Taber, K.S. (2018). The Use of Cronbach’s Alpha When Developing and Reporting Research Instruments in Science Education. *Research in Science Education*, 48(6), pp.1273–1296.
- Vaithilingam, P., Zhang, T., & Glassman, E. L. (2022). Expectation vs. experience: The gap between perceived and actual effectiveness of AI-powered code assistants. *CHI: Conference on Human Factors in Computing Systems*. <https://doi.org/10.1145/3491101.3519665>
- Venkatesh, V., Morris, M. G., Davis, G. B., & Davis, F. D. (2003). User acceptance of information technology: Toward a unified view. *MIS Quarterly*, 27(3), 425–478. <https://doi.org/10.2307/30036540>

12 Appendix

Additional supporting information including the survey instrument and responses are in the supplementary material, “Developer in Age of AI SUPPLEMENT”.

12.1 Statistical Analysis Support for RQ5

12.1.1 Elbow Graph to Determine Number of Clusters



Appendix Figure 1: Elbow Graph for RQ5

Variables used in WSS to define segment space:

- Intent to Increase Usage Index
- Strategic Outlook Index
- AI Coding Tool Index
- Perceived Quality Index (PQI)

12.1.2 Cluster Validation Analysis

To further evaluate the clustering solution, silhouette analysis was performed for candidate K-means solutions with $k = 2$ through $k = 5$.

The silhouette analysis indicated slightly higher scores for the two-cluster solution. However, the $k = 2$ solution produced overly broad groupings that reduced interpretability and obscured meaningful differences in AI usage, perceived outcomes, and adoption intent. The three-cluster solution provided a better balance between cluster separation and interpretability, while solutions with $k \geq 4$ did not yield substantively meaningful additional structure.

These results support the use of the three-cluster solution reported in the study.

Number of Clusters (k)	Silhouette Score
2	0.281
3	0.232
4	0.228

5	0.234
---	-------

Appendix Table 1: Silhouette Scores for Candidate Cluster Solutions

12.2 Additional Tables

These tables support sections in the paper.

Index Name	Description	Components (Summary)	Construction
Intent to Increase Usage Index	Measures likelihood of increased AI use	Coding + testing intent items	Mean
Perceived Quality Index (PQI)	Perceived impact of AI on code quality	Coding perceived quality + testing perceived quality	Mean
AI Coding Tool Index	Breadth of AI use in coding	Count of coding-related AI activities	Sum
AI Testing Tool Index	Breadth of AI use in testing	Count of testing-related AI activities	Sum
Strategic Outlook Index	Developers' beliefs about the future direction of AI-enabled software	Primary UI will be conversational, UI will be generated dynamically, Apps use multimodal input, Apps anticipate user need, Development status of AI Native apps, Agentic Architecture, RAG Relying on LLMs, Disposable Applications, SLMs, Hybrid Models LLMs with models of physical world, Benefits SLMs	Mean
Mean Concern Index	Perceived concerns of AI tool adoption	Security and compliance, cost of implementation and licensing, risk of copyright or IP infringement.	Mean

Appendix Table 2: Composite Indices Definitions

Variable Pair	N	ρ	p-value
Importance of Orchestration – Strategic Outlook	142	0.406	< 0.001
Importance of Prompt Engineering – Strategic Outlook	144	0.320	< 0.001
AI-Native Development Status – Strategic Outlook	144	0.427	< 0.001
Perceived productivity in coding – Strategic Outlook	145	0.242	0.003
Perceived productivity in testing – Strategic Outlook	132	0.249	0.004
Intent to Increase Usage – Strategic Outlook	147	0.305	< 0.001

Appendix Table 3: Associations with Strategic Outlook Index

Impact on AI-Native App Development	N	Mean	Median
-------------------------------------	---	------	--------

AI models hallucinations	140	3.84	4
Security risks	140	3.66	4
Difficulty in testing	141	3.6	4
Cost	141	3.51	4
Lack of skilled developers	141	3.39	3
Risk of betting on obsolete technologies	139	3.36	3
Complexity	140	3.29	3

Appendix Table 4: AI App Development